



# Algoritmos e Programação

Prof. Me. Fiorin  
andre.fiorin@iffarroupilha.edu.br



# Recapitulando

- Na última aula falamos sobre
  - Formas de representação de algoritmos
    - Descrição narrativa
    - Fluxogramas convencionais / diagrama de blocos
    - Diagrama de Chapin



# Aula 03

## Pseudocódigo



# Tipos primitivos de dados



# Pseudocódigo: tipos primitivos de dados

- Quando estamos construindo um algoritmo, geralmente é necessário armazenar temporariamente valores de diversos tipos.
  - Entrada
  - Atribuições e resultados de processamento
- Exemplo
  - Nome
  - Idade
  - Temperatura
  - Data
  - Resultado de um cálculo
  - ...



# Pseudocódigo: tipos primitivos de dados

- Esses dados podem ser classificados em 3 grupos:
  - **Dados numéricos**
    - Números inteiros
    - Números reais
  - **Dados literais**
    - Cadeias de caracteres
  - **Dados lógicos**
    - Valores booleanos



# Pseudocódigo: tipos primitivos de dados

- Tipos de dados – valores numéricos
  - **Inteiros:** Qualquer valor do conjunto dos números inteiros
    - Exemplos:
      - 54;
      - 0;
      - -18;
  - **Reais:** Qualquer valor do conjunto de números reais
    - Exemplos:
      - 3.35
      - 0.0
      - -12.075



# Pseudocódigo: tipos primitivos de dados

- Tipos de dados - Lógicos

- Também conhecidos como booleanos (*boolean*).

- Proveniente da lógica de Boole

- Exemplos

- 0 ou 1
    - Verdadeiro ou Falso (V ou F)
    - Sim ou Não
    - True ou False
    - Aberto ou Fechado





# Variáveis e constantes



# Pseudocódigo: variáveis e constantes

- Esses valores podem ser estáticos (não mudam no decorrer da execução do programa) ou dinâmicos (mudam durante a execução do programa).
- Assim, os valores manipulados em algoritmos podem ser:
  - **Constantes**
    - Exemplo:  $\pi = 3,14$
  - **Variáveis**
    - Exemplo: quantidade de um determinado item em estoque de uma loja.



# Pseudocódigo: variáveis e constantes

- **Variáveis**

- São usadas para armazenar valores dinâmicos.
- Devem possuir um nome único para identificação.
  - Dica: sempre usar nomes que facilitem o entendimento do código!
- Possuem um tipo de dado associado.
- Exemplos
  - `nota1: real;`
  - `nome: literal;`
  - `matricula: inteiro;`
  - `...`



# Pseudocódigo: variáveis e constantes

- **Constantes**

- Entidade que armazena um valor fixo, estável, inalterável ou imutável.
- Pode ser visto como uma “variável” que não pode ter o seu valor alterado.
- Devem possuir um nome único para sua identificação, de preferência em letras maiúsculas para diferenciar das variáveis.
- Exemplo
  - `CÓDIGO = 748;`
  - `PI = 3.14;`



# Pseudocódigo: variáveis e constantes

- **Importante**

- Os nomes de variáveis e constantes devem seguir regras simples:
  - Devem sempre começar com uma letra (a..z);
  - Os demais caracteres podem conter letras, números e *underline* (a..z, 0..9, \_);
  - Não devem possuir espaços

- **Exemplos**

- contador;
- nota1;
- media\_final;
- ~~media parcial;~~
- ~~1a\_nota;~~

**NÃO É  
PERMITIDO!!!**



# Estrutura de um algoritmo em português estruturado



# Pseudocódigo: estrutura do pseudocódigo

- Sintaxe

```
programa <nome_programa>  
const  
    <declaração de constantes>  
var  
    <declaração de variáveis>  
início  
    <lista de instruções ou comandos>  
    ...  
    ...  
    ...  
fim
```



# Pseudocódigo: estrutura do pseudocódigo

- Declaração de variáveis e constantes
  - Todos devem ser declarados no início do algoritmo

- **Sintaxe**

CONST

<NOME\_CONSTANTE> = <valor>;

VAR

<nome\_variável>: <tipo>;

- **Exemplo**

CONST

CODIGO\_DISCIPLINA = 7;

VAR

matricula: inteiro;

nome: literal[35];

temCelular: lógico;





# Pseudocódigo: estrutura do pseudocódigo

- Observações

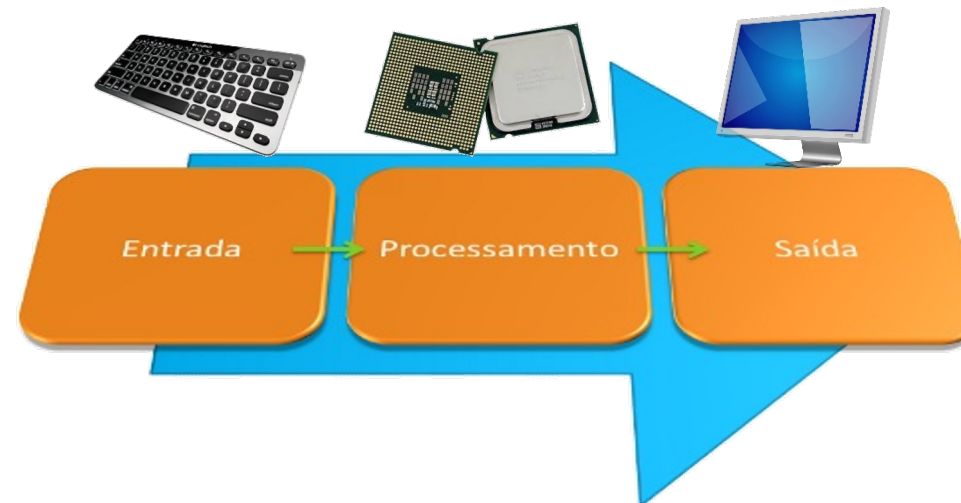
- A palavra **VAR** deverá estar presente e será utilizada uma única vez na definição de um conjunto de variáveis.
- Em uma mesma linha poderão ser definidas uma ou mais variáveis do mesmo tipo.
  - Cada variável em uma mesma linha deve ser separada por vírgula.
- Variáveis de tipos diferentes devem ser declaradas em linhas diferentes.
- Constantes são opcionais. Caso existam, devem ser declaradas após a palavra **CONST**.



# Comandos de entrada e saída

# Pseudocódigo: estrutura do pseudocódigo

- Os algoritmos possuem 3 passos básicos:
  1. Receber dados
  2. Processar dados a fim de gerar informações relevantes
  3. Gerar uma saída a partir dos dados processados





# Pseudocódigo: entrada e saída

- **Entrada de dados**

- São os dados recebidos pelo algoritmo que serão usados no processamento da informação.
- Comando específico para a leitura dos dados.
- Sintaxe
  - `leia (x, y);`
  - `leia (nota1);`
  - `leia (nome);`



# Pseudocódigo: entrada e saída

- Saída de dados

- Comando para exibir cadeias de caracteres, valores de variáveis e constantes, expressões, entre outros
  - Podem ser informações geradas a partir do processamento dos dados de entrada, palavras e/ou frases.
- Sintaxe

```
escreva (<lista de expressões>);
```
- Exemplos

```
escreva ("aprovado!");
escreva (x);
escreva (PI);
escreva ("média final", resultado);
```

**Observação 1:** *strings* devem ser apresentadas sempre entre aspas.

**Observação 2:** *strings* podem ser concatenadas com variáveis e constantes.



# Operadores



# Pseudocódigo: operadores

- **Operador de atribuição**

- É a principal maneira de armazenar um dado em uma variável.

- Exemplo

- `nome = "Oreste";`
    - `valor = 15.99;`
    - `quantidade = 5;`
    - `total = valor * quantidade;`
    - `imposto = total * 17 / 100;`



# Pseudocódigo: operadores

- **Operadores aritméticos**

- São responsáveis pelas operações matemáticas realizadas pelo computador.

| Nome             | Operador | Exemplo                           |
|------------------|----------|-----------------------------------|
| Soma             | +        | $5 + 4$                           |
| Subtração        | -        | $7 - 3$                           |
| Multiplicação    | *        | $2 * 10$                          |
| Divisão          | /        | $100 / 5$                         |
| Expoente         | ^        | $2 ^ 10$<br>(mesmo que $2^{10}$ ) |
| Resto de divisão | %        | $4 \% 2 ( = 0)$                   |





# Pseudocódigo: operadores

- **Prioridade dos operadores**

$$x = 10 - 7 * 2 + 0 * 4^2$$

$$x = ?$$

- a) 96
- b) -4
- c) 6

- **Ordem de prioridade**

- Parênteses
- Potências e raízes
- Multiplicações e divisões
- Adições e subtrações

- **Atenção:** operações de mesma prioridade são efetuadas da esquerda pra direita.



# Exemplo

# Pseudocódigo: Exemplo

- Exemplo de pseudocódigo

```

programa SomaDeInteiros
var
    x, y, soma: inteiro;
início
    escreva("Primeiro Número:");
    leia(x);
    escreva("Segundo Número:");
    leia(y);
    soma = x + y ;
    escreva ("A soma é: ", soma);
fim
  
```



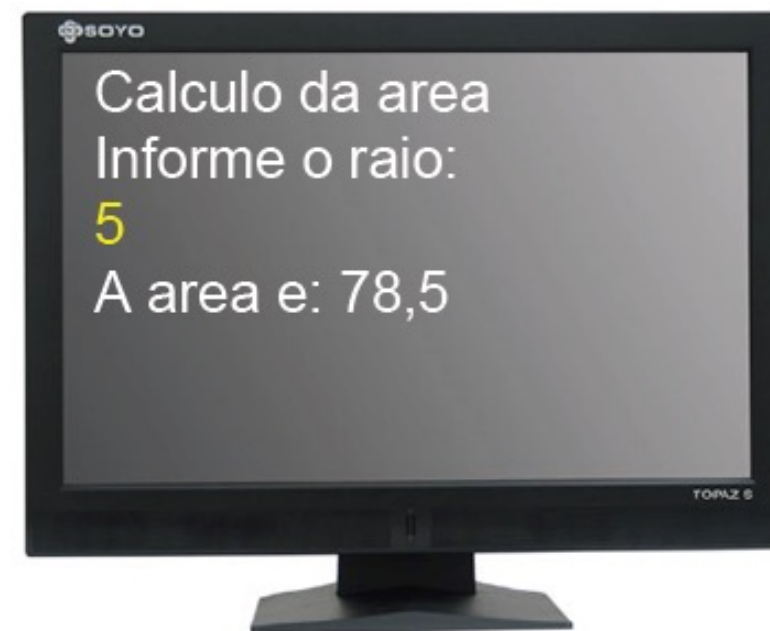
O ; (ponto-e-vírgula) vai no final de cada comando do algoritmo

# Pseudocódigo: exemplo

- Exemplo de pseudocódigo

```

programa CalculaArea
const
    PI = 3.14159;
var
    raio, area_circunferencia : real;
inicio
    escreva ("Calculo da área");
    escreva ("Informe o raio: ");
    leia (raio);
    area_circunferencia = PI * raio * raio;
    escreva ("A área é: ", area_circunferencia);
fim
    
```





# Exercícios



# Exercícios

1. Escreva um algoritmo que mostre na tela a mensagem “*Para aprender algoritmos é necessário desenvolver algoritmos*”.
2. Faça um algoritmo que receba duas notas de um aluno, calcule a média e apresente o resultado para o usuário.
3. Escreva um algoritmo em pseudocódigo que calcule a área de um triângulo e apresente o resultado.
  - Área =  $\frac{\text{base} \times \text{altura}}{2}$



# Na próxima aula...

- Desvios (testes) condicionais



# Referências

- Forbellone, A. L. **Lógica de programação – A construção de algoritmos e estruturas de dados**. 3ª edição, Pearson. 2007.